

▮ ДА! МСР ВСТРОЕНО В TENDERSHIELD (ОБЪЯСНЕНИЕ)

▮ КАК ЭТО РАБОТАЕТ (架構)

УРОВЕНЬ 1: САМА ПРОГРАММА TENDERSHIELD

TENDER SHIELD (React + FastAPI)
Frontend (React):
- UI для загрузки тендеров
- Отображение результатов
- Таблицы, графики, статистика
Backend (FastAPI):
- Deal Breaker Detector (regex)
- Financial Analyzer (расчёты)
- Opportunities Analyzer (Phase 2)
- API endpoints

УРОВЕНЬ 2: МСР - ДЛЯ РАЗРАБОТЧИКА (ТЫ)

CURSOR + 14 МСР (ИНСТРУМЕНТЫ)
Используются ДЛЯ РАЗРАБОТКИ:
- Code Quality - писать и тестировать
- Filesystem - создавать файлы
- Git - коммитить код
- Docker - запускать инфра
- Postgres - работать с БД
- Sequential Thinking - планировать
- Context7 - документация
- Playwright - E2E тесты
- Browser - проверка UI
- NPM - управление пакетами
+ НОВЫЕ:
- Qdrant (Vector DB)
- DuckDB (Data Processing)
- Text Analysis (фаза 2)
- API Integration (фаза 2)

УРОВЕНЬ 3: ИНФРАСТРУКТУРА (CONTAINER'Ы)

```
| DOCKER COMPOSE |  
| |  
| frontend - React приложение |  
| backend - FastAPI приложение |  
| postgres - основная БД |  
| qdrant - vector DB (НОВОЕ) |  
| (duckdb - встроен в backend) |
```

▣ КОНКРЕТНЫЙ ПРИМЕР: КАК ВСЁ РАБОТАЕТ ВМЕСТЕ

СЦЕНАРИЙ: Пользователь загружает тендер в TenderShield

1▣ ПОЛЬЗОВАТЕЛЬ ДЕЙСТВУЕТ:

- └─ Открывает <http://localhost:5173>
- └─ Нажимает "Загрузить тендер"
- └─ Выбирает PDF/текст

2▣ FRONTEND (React) ОТПРАВЛЯЕТ ЗАПРОС:

- └─ POST <http://localhost:8000/api/analyze>
- └─ С текстом тендера

3▣ BACKEND (FastAPI) ОБРАБАТЫВАЕТ:

- └─ Deal Breaker Detector анализирует текст (regex)
- └─ Financial Analyzer считает штрафы
- └─ Qdrant (Vector DB) ищет похожие тендеры
- └─ Postgres сохраняет результат
- └─ Возвращает JSON с результатами

4▣ FRONTEND ОТОБРАЖАЕТ:

- └─ Таблица deal breakers
- └─ Финансовая статистика
- └─ Рекомендация (АССЕРТ / SKIP)

КОГДА ИСПОЛЬЗУЮТСЯ MCP (ТЫ РАЗРАБОТЧИК):

СЦЕНАРИЙ: Нужно улучшить Deal Breaker Detector

1▣ ТЫ В CURSOR пишешь:

"Улучши regex для парсинга штрафов.
Используй Sequential Thinking для планирования"

2▣ CURSOR ИСПОЛЬЗУЕТ MCP:

- └─ Sequential Thinking - планирует как улучшить
- └─ Filesystem - редактирует файл [analyzers.py](#)
- └─ Code Quality - пишет тесты
- └─ Git - коммитит "feat: Improve penalty parsing"

- └─ Docker - перезагружает backend
- └─ Browser - открывает UI для проверки
- └─ Playwright - запускает E2E тесты

3▢ РЕЗУЛЬТАТ:

- └─ Улучшенный regex работает в TenderShield!

▢ АРХИТЕКТУРА: ГДЕ ЧТО ЖИВЁТ

tender-shield/

- |
- └─ frontend/ ← React приложение (ПОЛЬЗОВАТЕЛЬ видит)
 - | └─ src/components/
 - | └─ AnalysisTabs/ ← Результаты анализа выводятся тут
- |
- └─ backend/ ← FastAPI приложение (ГЛАВНЫЙ МОЗГ)
 - | └─ analyzers/
 - | └─ deal_breaker.py ← Detector (regex)
 - | └─ [financial.py](#) ← Financial analyzer
 - | └─ [opportunities.py](#) ← Phase 2
 - |
 - | └─ integrations/
 - | └─ qdrant_client.py ← Vector DB интеграция (НОВОЕ!)
 - | └─ duckdb_client.py ← Data Processing (НОВОЕ!)
 - | └─ egrul_api.py ← API Integration (фаза 2)
 - |
 - | └─ routes/
 - | └─ [analysis.py](#) ← API endpoints
- └─ docker-compose.yml ← Определяет контейнеры:
 - | └─ frontend service
 - | └─ backend service
 - | └─ postgres service (БД)
 - | └─ qdrant service (Vector DB) ← НОВОЕ!
 - | └─ (duckdb встроен в backend)
- └─ .cursorrules ← Правила для Cursor + MCP

▢ КРАТКИЙ ОТВЕТ: ДА или НЕТ?

❓ "MCP встроено в TenderShield?"

ДА, НО В ДВУХ РАЗНЫХ СМЫСЛАХ:

✔ СМЫСЛ 1: MCP ИСПОЛЬЗУЕТСЯ ДЛЯ РАЗРАБОТКИ TENDERSHIELD

Ты используешь MCP (через Cursor) чтобы:

- Писать код для TenderShield
- Тестировать код
- Создавать новые функции
- Деплоить приложение

Пример: Cursor с MCP помогает тебе написать Better Deal Breaker Detector → это встраивается в TenderShield → пользователи видят результат

Cursor + MCP → Разработка → TenderShield приложение → Пользователи

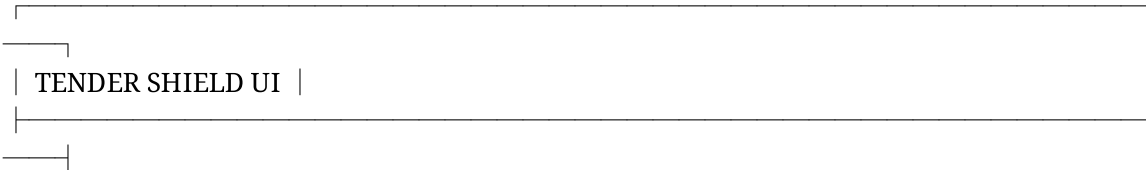
✔ СМЫСЛ 2: MCP СЕРВЕРЫ ЗАПУСКАЮТСЯ КАК ИНФРА ДЛЯ TENDERSHIELD

Некоторые MCP становятся частью инфраструктуры:

MCP	Запускает ся?	Зачем?	Видит ли пользователь?
Qdrant (Vector DB)	✔ ДА (docker)	Хранить нормативы	✗ НЕТ (backend использует)
DuckDB	✔ ДА (встроен)	Batch анализ	✗ НЕТ (backend использует)
Postgres	✔ ДА (docker)	Основная БД	✗ НЕТ (backend использует)
Sequential Thinking	✗ НЕТ	Курсор планирует	✗ НЕТ (ты используешь)
Git	✗ НЕТ	Курсор коммитит	✗ НЕТ (ты используешь)
Code Quality	✗ НЕТ	Курсор тестирует	✗ НЕТ (ты используешь)

▮ КОНЕЧНЫЙ РЕЗУЛЬТАТ ДЛЯ ПОЛЬЗОВАТЕЛЯ

Пользователь TenderShield видит:



Загрузи тендер → PDF/текст

[Выбрать файл]

РЕЗУЛЬТАТЫ АНАЛИЗА:

▢ DEAL BREAKERS (из Deal Breaker Detector + Text Analysis MCP):

• Штраф 1,000,000 руб

• Неактуальный ГОСТ

▢ ФИНАНСОВЫЕ УСЛОВИЯ (из Financial Analyzer + DuckDB):

• Средний штраф: 500,000 руб

• Диапазон: 10,000 - 5,000,000

▢ ПОХОЖИЕ ТЕНДЕРЫ (из Qdrant Vector DB):

• 3 похожих тендера найдено

▢ ПРОВЕРКА КОМПАНИИ (из API Integration - Phase 2):

• Статус: АКТИВНА ✓

• Налоги: В ПОРЯДКЕ ✓

• Риск: НИЗКИЙ ✓

✓ РЕКОМЕНДАЦИЯ:

"УЧАСТВОВАТЬ - хорошие условия"

**Пользователь НЕ видит что это:

- Deal Breaker Detector (regex)
- Qdrant (Vector DB)
- DuckDB (Data Processing)
- API Integration MCP

Он видит просто **результаты анализа!** ☆☆☆

▢ ИТОГОВЫЙ ОТВЕТ

"MCP встроено в TenderShield?"

УРОВЕНЬ 1 (РАЗРАБОТКА):

✓ ДА - Cursor + 14 MCP используются для разработки TenderShield
(ты пишешь код с помощью MCP)

УРОВЕНЬ 2 (ИНФРАСТРУКТУРА):

✓ ЧАСТИЧНО - Qdrant + DuckDB запускаются как контейнеры
(они поддерживают работу TenderShield)

УРОВЕНЬ 3 (ПОЛЬЗОВАТЕЛЬ):

✗ НЕТ - Пользователь видит только результаты
(ему не важно как это работает внутри)

ПРОЩЕ:

**MCP - это ИНСТРУМЕНТЫ для разработки и инфраструктура для работы,
а TenderShield - это ПРОДУКТ который пользователь видит.**

▮ АНАЛОГИЯ

MCP ~ Кухня в ресторане

TenderShield ~ Блюдо которое видит клиент

Клиент не видит кухню, не видит инструменты повара,
но видит вкусное блюдо (результаты анализа)!

✓ CHECKLIST ПОНИМАНИЯ

- [] MCP используются для РАЗРАБОТКИ TenderShield (Cursor помогает писать код)
- [] Некоторые MCP сервера (Qdrant, DuckDB) запускаются как инфра
- [] Пользователь не видит MCP - видит только результаты
- [] TenderShield = готовый продукт который использует MCP'ы внутри
- [] Это как iPhone - пользователь не видит как работает железо, видит только UI

ГОТОВО! ТЕПЕРЬ ТЫ ПОНИМАЕШЬ ПОЛНУЮ АРХИТЕКТУРУ! ▮